

AI SCIENTIST

FASE 2 | AULA 02 -

SQL PARA ANÁLISE AVANÇADA: AGREGAÇÕES E SUBQUERIES

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 5

MERCADO, CASES E TENDÊNCIAS 24

O QUE VOCÊ VIU NESTA AULA? 25

REFERÊNCIAS 26

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VEM POR AÍ?

Nesta aula, sairemos do básico SELECT * FROM e entraremos na camada lógica de transformação. Você aprenderá como o SQL pode realizar 80% do seu trabalho de Feature Engineering e limpeza de dados antes mesmo de abrir um DataFrame. Vamos dominar o agrupamento de dados, filtros pós-agregação e a construção de consultas complexas através de subqueries e CTEs (Common Table Expressions).

Até agora, tratamos o banco de dados como um repositório de onde extraímos dados brutos. Nesta aula, faremos uma mudança de paradigma: **pararemos de apenas "puxar" dados para começar a processar inteligência diretamente no motor SQL.**

Em cenários de Big Data, transferir gigabytes de dados para um notebook Python é ineficiente. O SQL analítico permite que você realize transformações complexas, agregações estatísticas e cruzamentos de níveis lógicos distintos (via subqueries) sem que os dados saiam do servidor.

Ao final desta aula, você será capaz de construir consultas que resolvem problemas de negócio complexos — como o cálculo de LTV, análise de Churn e segmentação de clientes — de forma performática e modular.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

HANDS ON

Nossa prática será baseada em um cenário de E-commerce. Nós iremos calcular o Lifetime Value (LTV) de clientes usando GROUP BY, identificar produtos "estrela" e produtos "estoque parado" com filtros HAVING, criar rankings de vendas usando subqueries, organizar scripts complexos de análise de churn utilizando CTEs.

Além disso, também temos um desafio prático: Quem são os 5% dos clientes que geraram 50% do faturamento no último trimestre?

Utilizaremos também o banco de dados **"GlobalTech E-commerce"**. O foco não é apenas a sintaxe, mas a resolução de problemas reais de um(a) Cientista de Dados. Cheque a estrutura do Lab no início da seção “Saiba mais”.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SAIBA MAIS

Estrutura do Lab:

- **Tabelas de Trabalho:** clientes, pedidos, itens_pedido, produtos e categorias.
- **Cenário 1 (Agregações):** identificar o ticket médio por categoria de produto, filtrando apenas categorias que faturaram mais de R$ 50.000 no último mês.
- **Cenário 2 (Subqueries):** listar todos os clientes cujas compras individuais foram, em média, 20% superiores à média global de vendas da empresa.
- **Cenário 3 (CTEs):** criar uma tabela temporária lógica para calcular o tempo médio entre a primeira e a segunda compra de cada cliente (Análise de Retenção).

Agregações: resumindo o caos

As funções de agregação são o fundamento da análise descritiva. No entanto, o comportamento delas em relação a valores NULL e à performance é onde o(a) especialista se diferencia. As funções de agregação (COUNT, SUM, AVG, MIN, MAX) são os primeiros passos estatísticos. No MySQL, a performance dessas funções depende de como o GROUP BY interage com os índices.

O filtro de agregados: a diferença fundamental entre WHERE (filtra linhas antes da conta) e HAVING (filtra o resultado da conta).

Comportamento de NULLs: entender que COUNT(*) conta linhas, enquanto COUNT(coluna) ignora nulos. Por que AVG(coluna) pode ser estatisticamente perigoso se você não tratar os nulos previamente (com COALESCE, por exemplo).

DISTINCT inside Aggregations: o uso de COUNT(DISTINCT cliente_id) para métricas de usuários únicos, e o custo computacional disso em tabelas de bilhões de linhas.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

A DIFERENÇA FUNDAMENTAL NO COUNT

No SQL, o NULL não é um valor (como zero ou uma string vazia); ele é a **ausência de valor** ou um estado desconhecido. Por isso, as funções de agregação o tratam de formas distintas:

- **COUNT(*)**: ele olha para a **linha** como um todo. Se a linha existe, ele a conta, independentemente de haver campos nulos nela. É o comando para saber "quantos registros existem nesta tabela".
- **COUNT(coluna)**: ele olha especificamente para o conteúdo daquela coluna. Se encontrar um NULL, ele ignora aquela linha na contagem.

Exemplo prático

Imagine uma tabela de Vendas com 4 linhas:

ID	VALOR
1	100
2	200
3	NULL
4	300

Tabela 1: Tabela de vendas
Fonte: Elaborado pelo autor (2026)

COUNT(*) resultará em **4**, enquanto COUNT(Valor) resultará em **3**.

POR QUE O AVG(COLUNA) É "PERIGOSO"?

O perigo do AVG (média) reside na matemática interna da função. A fórmula da média aritmética é:

$$Média = \frac{Soma\ dos\ Valores}{Quantidade\ de\ Itens}$$

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O SQL segue essa lógica, mas ele ignora os NULLs tanto no numerador quanto no denominador.

O cenário do erro: imagine que você está analisando o gasto médio de clientes em uma loja. Você tem 10 clientes. 5 clientes gastaram **R$100** cada e 5 clientes **não compraram nada** (e, por um erro de *design* ou *input*, o sistema registrou isso como NULL em vez de 0).

O que o SQL faz com AVG (gasto): ele ignora os 5 nulos. A conta vira:

$$\frac{500}{5} = 100$$

Resultado: "Seu gasto médio por cliente é R$100".

O que a realidade diz: Você tem 10 clientes e o total arrecadado foi R$500.

$$\frac{500}{10} = 50$$

Resultado Real: "Seu gasto médio por cliente é R$50".

Logo, o perigo é a **inflação artificial dos números**. Se você ignora os nulos, você está trabalhando apenas com a média de quem realizou uma ação, e não com a média do grupo total. Em relatórios financeiros ou de performance, isso pode mascarar um problema grave de engajamento ou conversão, fazendo o resultado parecer muito melhor do que realmente é.

Como se proteger com COALESCE

Para evitar esse viés, você deve decidir o que o NULL representa. Se o nulo deveria ser interpretado como zero, você usa o COALESCE.

O COALESCE(coluna, 0) transforma o NULL em 0 **antes** da função de média processar o dado.

A forma segura:

```
SELECT
   AVG(COALESCE(valor, 0)) AS media_correta
FROM Vendas;
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Nesse caso, o SQL tratará os nulos como zeros, incluindo-os no denominador e trazendo a média para a realidade estatística.

Resumindo:

- **COUNT(*)**: conta tudo.
- **COUNT(coluna)**: ignora NULLs.
- **AVG(coluna)**: ignora NULLs (perigo de inflar a média).
- **COALESCE**: é o seu escudo para garantir que o "nada" seja contado como "zero".

Assim fica clara a diferença de como o SQL "enxerga" esses espaços vazios! Verifique também como isso afeta outras funções, como o SUM ou o MAX.

GROUP BY: A ENGENHARIA DE AGRUPAMENTO

O GROUP BY é, sem dúvida, a ferramenta mais poderosa para transformar **dados brutos** em **informação estratégica**. Se o SQL fosse uma câmera, o GROUP BY seria o ajuste do **foco** e do **zoom**: ele define o que você está enxergando e com que nível de detalhe.

Aqui está a explicação detalhada sobre como ele molda a sua análise:

O Conceito de Granularidade

Granularidade refere-se ao nível de detalhe dos seus dados. **Baixa Granularidade significa:** visão macro (Ex.: Total de vendas da empresa no ano) **e Alta Granularidade:** Visão micro (Ex.: Total de vendas por produto, por hora, em cada filial).

Quando você usa o GROUP BY, você está "achatando" a tabela. Se você tem 1 milhão de linhas de vendas e agrupa por Estado, o resultado terá apenas 27 linhas (no Brasil). Você trocou o detalhe da transação individual pela visão regional.

Múltiplas Dimensões (Séries Temporais e Hierarquias)

O verdadeiro poder surge quando você combina várias colunas. Isso permite cruzar informações para encontrar padrões que uma única dimensão esconderia.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Exemplo de visão em série temporal:

Se você agrupar apenas por **Categoria**, verá qual vende mais. Mas se agrupar por **Ano**, **Mês** e **Categoria**, você verá a **sazonalidade**.

Ano	Mês	Categoria	Total Vendido
2023	01	Eletrônicos	R$50.000
2023	01	Roupas	R$20.000
2023	02	Eletrônicos	R$45.000

Tabela 2: Exemplo de visão em série temporal
Fonte: Elaborado pelo autor (2026)

O que acontece aqui? O SQL cria um "balde" único para cada combinação distinta de Ano+Mês+Categoria. Se um desses elementos mudar, uma nova linha de agrupamento é criada.

Expressões no agrupamento: criando dimensões "On-the-fly"

Muitas vezes, as categorias que você precisa para análise não existem fisicamente na tabela. É aqui que entra o agrupamento por expressões ou lógica (CASE WHEN). Em vez de agrupar por colunas fixas, você agrupa pelo resultado de uma lógica.

Exemplo: faixas de preço

Imagine que você quer saber quantas vendas teve em produtos "Baratos" vs. "Caros", mas não tem essa coluna na tabela.

```
SELECT
   CASE
      WHEN preco < 50 THEN 'Econômico'
      WHEN preco BETWEEN 50 AND 200 THEN 'Standard'
      ELSE 'Premium'
   END AS categoria_preco,
   COUNT(*) AS qtd_vendas,
   SUM(preco) AS receita_total
FROM Vendas
GROUP BY 1; -- O '1' refere-se à primeira coluna (a expressão CASE)
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Por que isso é incrível?

- Você não precisa pedir para o DBA criar uma nova coluna na tabela;
- Você pode testar diferentes hipóteses de segmentação de mercado instantaneamente;
- Você pode agrupar por datas formatadas (Ex.: GROUP BY DATE_TRUNC('week', data_venda)) para ver o desempenho semanal.

A regra de ouro do GROUP BY

Existe uma regra técnica que causa muitos erros em iniciantes: **"Toda coluna que estiver no seu SELECT e não for uma função de agregação (SUM, AVG, COUNT, etc.), DEVE obrigatoriamente estar no seu GROUP BY.".**

Se você tentar selecionar o nome do cliente, mas agrupar apenas pela data, o SQL não saberá qual nome de cliente exibir para aquele grupo (já que podem existir vários) e retornará um erro. **Resumo:**

- **Sem GROUP BY:** você vê a floresta árvore por árvore (muito detalhe, pouca síntese).
- **Com GROUP BY:** você vê os bosques e setores da floresta (padrões e totais).
- **Com expressões:** você "pinta" as árvores de cores diferentes antes de contar (análise personalizada).

Imagine que temos uma tabela chamada pedidos. Nem todo registro ali é uma venda concluída; alguns podem ser **carrinhos abandonados** ou **leads**, onde o valor_total está como NULL.

O cenário: queremos um relatório anual que segmente os clientes em duas categorias: **"VIP"** (compras acima de 500) e **"Varejo"** (abaixo de 500). Precisamos saber o ticket médio real, considerando que um carrinho abandonado (NULL) conta como **0 reais** para a média da empresa.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
SELECT
   -- 1. Definindo a granularidade temporal (Ano)
   EXTRACT(YEAR FROM data_pedido) AS ano,
   -- 2. Criando uma dimensão on-the-fly (Segmentação)
   CASE
      WHEN valor_total > 500 THEN 'Perfil VIP'
      ELSE 'Perfil Varejo'
   END AS perfil_cliente,
   -- 3. O perigo dos NULLs: COUNT(*) vs COUNT(coluna)
   COUNT(*) AS total_interacoes, -- Conta tudo (inclusive carrinhos vazios)
   COUNT(valor_total) AS vendas_realizadas, -- Ignora NULLs (vendas efetivas)
   -- 4. A média estatisticamente segura (Tratando NULL como 0)
   AVG(COALESCE(valor_total, 0)) AS ticket_medio_real,
   -- 5. A média "perigosa" (Para comparação)
   AVG(valor_total) AS ticket_medio_inflado

FROM pedidos
GROUP BY
   EXTRACT(YEAR FROM data_pedido),
   CASE
      WHEN valor_total > 500 THEN 'Perfil VIP'
      ELSE 'Perfil Varejo'
   END
ORDER BY ano DESC, perfil_cliente;
```

Por que essa consulta é uma aula de SQL? A Anatomia do Agrupamento:

Note que repetimos a lógica do CASE WHEN no GROUP BY. Isso diz ao SQL: "Primeiro calcule quem é VIP e quem é Varejo, depois crie as caixinhas de grupo". A **granularidade** aqui é dupla: por **Tempo** (Ano) e por **Comportamento** (Perfil).

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O "Pulo do Gato" nos cálculos:

1. **total_interacoes** vs **vendas_realizadas**: se o resultado mostrar 100 interações e 80 vendas, você sabe instantaneamente que teve 20% de abandono de carrinho. Se usasse apenas COUNT(*), você estaria no escuro sobre a conversão.
2. **ticket_medio_real**: ao usar COALESCE(valor_total, 0), você força o SQL a incluir os desistentes na conta. A média cai, mas ela reflete a **realidade financeira** da operação.
3. **ticket_medio_inflado**: esta coluna existiria apenas para te mostrar o erro. Ela ignoraria os NULLs, fazendo parecer que seus clientes gastam mais do que realmente gastam no consolidado.

ano	perfil_cliente	total_interacoes	vendas_realizadas	ticket_medio_real	ticket_medio_inflado
2024	Perfil VIP	100	100	750.00	750.00
2024	Perfil Varejo	200	150	112.50	150.00

Tabela 3: Perfil
Fonte: Elaborado pelo autor (2026)

Análise: no "Perfil Varejo", o ticket inflado diz que a média é 150. Mas como 50 pessoas não compraram nada (NULL), a média real de receita por interessado é 112,50. **Essa diferença de 25% é o que quebra o planejamento de uma empresa.**

WHERE VS. HAVING: A ORDEM LÓGICA DE PROCESSAMENTO

Este é o conceito que separa quem "escreve" SQL de quem realmente **entende** o que o banco de dados está fazendo por trás das cortinas.

O grande segredo aqui é que a ordem em que você **escreve** o código é totalmente diferente da ordem em que o banco de dados o **executa**.

Um erro comum é tentar filtrar resultados agregados no WHERE.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

A ordem de execução: FROM → WHERE (filtra linhas) → GROUP BY → HAVING (filtra grupos) → SELECT → ORDER BY.

Essa ordem é eficiente pois você deve filtrar o máximo possível no WHERE antes de deixar o banco realizar o trabalho pesado de agrupamento.

A ordem lógica de execução (O "mapa" do SQL)

Imagine que o banco de dados é um funcionário de um armazém. Se você pede um relatório, ele segue estes passos rígidos:

1. **FROM:** ele vai até a prateleira (tabela).
2. **WHERE:** ele descarta os itens estragados ou que não interessam antes de pegá-los (filtra linhas individuais).
3. **GROUP BY:** ele separa os itens restantes em caixas (agrupa).
4. **HAVING:** ele olha para as caixas prontas e descarta as que não atingem um critério (filtra grupos).
5. **SELECT:** ele escreve no papel apenas as informações que você pediu.
6. **ORDER BY:** ele organiza as linhas do papel final.

WHERE VS. HAVING: A BATALHA DO FILTRO

O erro mais comum é tentar fazer isso: SELECT categoria, SUM(valor) FROM vendas WHERE SUM(valor) > 1000 ... **ERRO! Mas por que isso falha?**

Porque no passo 2 (WHERE), o banco de dados ainda não sabe qual é a soma (SUM). Ele ainda está olhando para cada linha individual. A "caixa" (o grupo) ainda não foi montada.

- **O WHERE filtra a "matéria-prima"** (as linhas da tabela);
- **O HAVING filtra o "produto acabado"** (os resultados das funções de agregação).

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

A ENGENHARIA DA EFICIÊNCIA: FILTRE CEDO, FILTRE RÁPIDO

Aqui entra a parte de performance. Imagine uma tabela com 100 milhões de vendas de 10 anos de história. Você quer saber quais categorias venderam mais de R$50k **apenas no ano de 2023.**

Abordagem lenta (ou impossível, dependendo da lógica): tentar processar tudo e deixar para filtrar no final. Se você não usar o WHERE para o ano, o banco terá que:

1: Ler 100 milhões de linhas.
2: Agrupar 100 milhões de linhas.
3: Só então descartar os grupos dos anos anteriores no HAVING.

Abordagem eficiente (The SQL Way):

```
SELECT
   categoria,
   SUM(valor_total) AS total_vendas
FROM vendas
WHERE data_venda >= '2023-01-01' -- FILTRO DE LINHA: O banco já ignora 90% dos dados aqui
GROUP BY categoria
HAVING SUM(valor_total) > 50000; -- FILTRO DE GRUPO: Só as categorias que bateram a meta
```

A regra de ouro: tudo o que puder ser filtrado por colunas simples (datas, status, IDs) deve estar no WHERE. Deixe para o HAVING apenas o que depende de cálculos (SUM, AVG, COUNT).

Exemplo prático: o filtro de clientes ativos

Imagine que você quer a lista de clientes que fizeram mais de 5 compras, mas quer considerar apenas compras não canceladas.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Passo	Comando	O que o banco faz
De onde?	FROM pedidos	Abre a tabela de pedidos.
Qual critério?	WHERE status != 'Cancelado'	Descarta todas as linhas canceladas. Menos dados para o resto do processo.
Como agrupar?	GROUP BY cliente_id	Cria as "caixas" por cliente.
Qual meta?	HAVING COUNT(*) > 5	Filtra as caixas: Se o cliente tem 4 pedidos, a caixa dele é jogada fora agora.

Tabela 4: Filtro de clientes
Fonte: Elaborado pelo autor (2026)

Por que isso é importante para você?

Se você colocar um filtro que poderia estar no WHERE dentro do HAVING, sua consulta pode demorar minutos em vez de segundos. O banco de dados é forçado a fazer um trabalho pesado de agrupamento para algo que ele poderia ter descartado logo no início.

Entender essa ordem de processamento ajuda muito quando você começa a usar Alias (apelidos de colunas). Você sabia que, em muitos bancos de dados, você não pode usar o apelido criado no SELECT dentro do WHERE?

SUBQUERIES: CONSULTAS DENTRO DE CONSULTAS

As subqueries são como as "bonecas russas" (matrioskas) do SQL: você coloca uma consulta dentro da outra para resolver problemas complexos em etapas. Elas são extremamente elegantes, mas se você não souber o que está acontecendo "sob o capô", pode acabar criando uma consulta que trava o banco de dados. Vamos dissecar os três níveis:

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Subqueries escalares (O resultado único)

Uma subquery escalar é aquela que retorna **exatamente um valor** (uma única célula: 1 linha e 1 coluna). Ela é comumente usada no WHERE ou no SELECT.

O caso de uso: "Quero ver todos os produtos que custam mais que a média da loja.".

A lógica: primeiro o SQL calcula a média (Ex.: 50.00) e depois substitui esse valor na consulta principal.

```
SELECT nome, preco
FROM produtos
WHERE preco > (SELECT AVG(preco) FROM produtos);
```

Subqueries correlacionadas (O "vilão" da performance)

Aqui é onde mora o perigo. Diferente da escalar, que executa uma vez e pronto, a subquery correlacionada **depende de valores da consulta externa** para funcionar.

Como funciona: o banco de dados lê a primeira linha da tabela externa, "pergunta" algo para a subquery usando um dado dessa linha, recebe a resposta e passa para a segunda linha.

O problema: se a tabela externa tem 1 milhão de linhas, a subquery será executada **1 milhão de vezes**.

Exemplo: "Encontrar funcionários que ganham mais que a média de seus próprios departamentos."

```
SELECT nome, salario, departamento_id
FROM funcionarios f_externo
WHERE salario > (
   SELECT AVG(salario)
   FROM funcionarios f_interno
   WHERE f_interno.departamento_id = f_externo.departamento_id -- O elo de
ligação
);
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Dica de otimização: muitas vezes, uma subquery correlacionada pode (e deve) ser substituída por um **JOIN** ou uma **Window Function** (OVER(PARTITION BY...)), que são processados de forma muito mais eficiente pelo motor do banco.

OPERADORES DE CONJUNTO: O DEBATE IN VS. EXISTS

Essa é uma das discussões mais clássicas em tuning de SQL. Ambos servem para verificar se um dado existe em outra lista, mas a estratégia de busca é diferente.

O Operador IN

O IN é ideal para listas pequenas e estáticas. Ele funciona como um "OR" gigante. WHERE id IN (1, 2, 3) é o mesmo que WHERE id = 1 OR id = 2 OR id = 3.

Ponto fraco: ele processa a subquery inteira antes de comparar. Além disso, o NOT IN tem um comportamento perigoso: se a subquery retornar um único NULL, a consulta inteira pode retornar zero resultados.

O Operador EXISTS

O EXISTS é um teste booleano (Verdadeiro/Falso). Ele não se importa com os dados, apenas se algo foi encontrado.

A vantagem: ele usa uma estratégia de "curto-circuito". Assim que o SQL encontra a primeira linha que satisfaz a condição, ele para de procurar e retorna "Verdadeiro".

Performance: em tabelas grandes, o EXISTS costuma ganhar do IN, especialmente se a coluna estiver indexada.

Operador	Melhor Uso	Comportamento com NULL
IN	Listas pequenas ou fixas	Sensível (pode quebrar a lógica).
EXISTS	Tabelas grandes / Subqueries correlacionadas	Seguro (ignora o conteúdo, foca na existência).

Tabela 5: NULL
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

POR QUE AS SUBQUERIES PODEM SER PERIGOSAS?

O perigo não está no resultado, mas no **custo computacional**.

I/O de Disco: subqueries mal escritas podem forçar o banco a ler os mesmos dados milhares de vezes.

Falta de índices: se a coluna usada para ligar a consulta externa à interna (na correlacionada) não estiver indexada, o tempo de execução cresce exponencialmente.

Legibilidade: consultas com 4 ou 5 níveis de subqueries tornam-se "Dívida Técnica" — ninguém consegue dar manutenção ou entender a lógica meses depois.

Resumo de otimização:

- Use **Subqueries Escalares** para comparações simples;
- Evite **correlacionadas** em tabelas gigantes; prefira JOIN;
- Prefira **EXISTS** em vez de IN quando estiver lidando com volumes de dados consideráveis.

As **Window Functions** (Funções de Janela) são a evolução natural das subqueries correlacionadas. Elas permitem que você olhe para um grupo de linhas (a "janela") sem precisar "achatar" a tabela com um GROUP BY e sem a lentidão de uma subquery que executa linha por linha.

O DESAFIO: A MÉDIA POR DEPARTAMENTO

Lembra do problema: "Mostrar funcionários que ganham mais que a média de seus próprios departamentos"?

A abordagem lenta (Subquery correlacionada):

O banco de dados precisa recalcular a média do departamento para cada funcionário da tabela. É um processo repetitivo.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

A abordagem moderna (Window Function):

```
SELECT
    nome,
    departamento_id,
    salario,
    AVG(salario) OVER(PARTITION BY departamento_id) AS media_depto
FROM funcionarios;
```

Por que isso é melhor?

Performance (Otimização de varredura): o banco de dados faz uma única passagem pelos dados. Ele organiza os departamentos na memória, calcula as médias e as "anexa" às linhas originais. Não há reexecução de subqueries.

Clareza: você mantém todos os detalhes da linha (nome, cargo, salário) e a agregação (média) lado a lado, sem complicação.

O toque final: filtrando com a janela

Um detalhe importante: você não pode usar uma Window Function diretamente no WHERE (pela mesma ordem de execução que vimos antes: o WHERE vem antes da "janela" ser aberta).

Para filtrar os funcionários que ganham acima dessa média, usamos uma CTE (Common Table Expression), que deixa seu código limpo e organizado:

```
WITH CalculoMedias AS (
    SELECT
        nome,
        salario,
        departamento_id,
          AVG(salario) OVER(PARTITION BY departamento_id) AS media_depto
      FROM funcionarios
)
SELECT * FROM CalculoMedias
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
WHERE salario > media_depto; -- Agora o filtro é simples e rápido!
```

Por que a CTE é superior à Subquery aqui?

- **Legibilidade:** o código é lido de cima para baixo, como uma receita. Primeiro você define o que quer calcular (WITH), depois você consome esse resultado.
- **Reutilização:** se você precisasse fazer mais cálculos com essa media_depto na mesma consulta, você poderia, sem ter que escrever a subquery novamente.

CTES (COMMON TABLE EXPRESSIONS): O SQL MODERNO

Introduzidas no MySQL 8.0, as CTEs (WITH) substituem subqueries complexas, tornando o código legível e modular. Para um(a) Cientista de Dados, uma CTE é como definir uma variável ou função, permitindo que outros colegas entendam sua lógica de análise.

As **CTEs (Common Table Expressions)**, representam a transição do SQL "macarrônico" para o SQL estruturado e profissional. Se as subqueries são como notas de rodapé que interrompem a leitura, as CTEs são como os capítulos de um livro: elas contam a história da sua análise passo a passo.

Do "Código em Cebola" para o "Código em Blocos"

Antes das CTEs, para fazer análises complexas, você criava subqueries aninhadas. Para entender o código, você precisava começar do centro (a consulta mais interna) e ir "descascando" para fora.

O problema (Subqueries aninhadas):

```
SELECT ... FROM (SELECT ... FROM (SELECT ... FROM tabela) AS sub1) AS
sub2; -- Ilegível!
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

A Solução (CTE): com a CTE, você isola a lógica. É como se você dissesse: "Primeiro, prepare este conjunto de dados e chame-o de 'VendasLimpas'. Depois, use o 'VendasLimpas' para calcular isso."

MODULARIDADE: O SQL COM "MENTALIDADE DE PROGRAMADOR"

Para um(a) Cientista de Dados, a CTE funciona como a definição de uma **variável de tabela**. Você pode encadear várias CTEs, onde uma consulta utiliza o resultado da anterior.

Vejamos um exemplo de fluxo modular. Imagine que você precisa:

1. Limpar os dados.
2. Calcular a média por usuário.
3. Filtrar apenas quem está acima da média.

```
WITH dados_limpos AS (
    -- Etapa 1: Tratamento de nulos e filtros básicos
    SELECT id_usuario, COALESCE(valor, 0) AS valor_ajustado
    FROM vendas
    WHERE status = 'concluido'
),
metricas_usuario AS (
    -- Etapa 2: Agregação (usa o resultado da primeira CTE)
    SELECT id_usuario, SUM(valor_ajustado) AS total_gasto
    FROM dados_limpos
    GROUP BY id_usuario
)
-- Consulta Final: Onde tudo se une
SELECT * FROM metricas_usuario
WHERE total_gasto > (SELECT AVG(total_gasto) FROM metricas_usuario);
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

AS VANTAGENS PRÁTICAS NO DIA A DIA

Debugging (Depuração) facilitada

Em uma subquery complexa, se o resultado estiver errado, você tem que desmontar a query inteira. Com CTEs, você pode simplesmente comentar a consulta final e fazer um SELECT * FROM dados_limpos para verificar se a primeira etapa está correta. Isso economiza horas de trabalho.

Manutenção e revisão por pares (Code Review)

Se um colega precisar revisar seu código, ele lerá os nomes dos blocos (dados_limpos, metricas_usuario) e entenderá sua intenção imediatamente. A CTE documenta o processo de pensamento do(a) analista enquanto o código é escrito.

Reutilização no mesmo Escopo

Diferente de uma subquery, que você teria que escrever duas vezes se precisasse dela em um JOIN e em um WHERE, a CTE é definida uma única vez no topo e pode ser chamada múltiplas vezes ao longo da mesma query.

ONDE A CTE BRILHA (RECORRÊNCIA)

Embora existam as **CTEs Recursivas** (usadas para hierarquias como organogramas ou árvores de decisão), o uso mais comum para Ciência de Dados é o **Refactoring**. Ela transforma scripts de 200 linhas de puro caos em blocos lógicos que qualquer pessoa da equipe consegue manter.

Característica	Subqueries Tradicionais	CTEs (WITH)
Legibilidade	Baixa (lógica de dentro para fora)	Alta (lógica de cima para baixo)
Manutenção	Difícil e propensa a erros	Modular e fácil de testar
Reutilização	Precisa repetir o código	Definida uma vez, usada várias

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Organização	"Ninhos" de código	Blocos nomeados (variáveis)

Tabela 6: CTEs
Fonte: Elaborado pelo autor (2026)

Ficamos por aqui nessa jornada pelos pilares do SQL! Agora que você domina desde o comportamento dos NULLs até a elegância das CTEs, você tem a base completa para escrever queries que não são apenas corretas, mas também performáticas e profissionais.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

MERCADO, CASES E TENDÊNCIAS

O papel do(a) Analytics Engineer

O mercado criou uma nova função entre o(a) Engenheiro(a) de Dados e o(a) Cientista de Dados: o(a) **Analytics Engineer**. Este(a) profissional vive de SQL Avançado, transformando dados brutos em modelos prontos para o BI.

Case de Sucesso: como a **Netflix** utiliza SQL analítico para pré-processar trilhas de visualização antes de alimentar modelos de recomendação.

Tendência: dbt (data build tool). O mercado está migrando para o "Analytics Engineering", onde o SQL é a linguagem principal para transformar dados brutos em tabelas prontas para consumo (Business Ready). Ela utiliza de CTEs para criar pipelines de dados robustos. Entender esse conceito é o requisito #1 para trabalhar com dbt.

O fim do processamento local: com o aumento dos volumes de dados, processar 1 TB de dados no banco (via SQL) é infinitamente mais barato e rápido do que baixar para um arquivo local.

SQL em Data Warehouses Modernos: ferramentas como BigQuery e Snowflake focam em "SQL-first". O conhecimento que você adquire aqui no MySQL é 95% transferível para essas plataformas de escala petabyte.

Tendência: SQL for Real-time: o uso de SQL analítico em fluxos de dados em tempo real (como o KSQL ou Flink SQL) para detecção de fraudes instantânea.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Agregação não é só soma: é a base para KPIs. A atenção aos nulos e à granularidade (GROUP BY) define a precisão do seu indicador.

A Lógica do Filtro: WHERE limpa o dado bruto; HAVING limpa o indicador calculado.

Complexidade Modular: subqueries resolvem problemas de múltiplos níveis, mas as **CTEs** são o padrão ouro para código limpo e escalável em equipes de alto desempenho.

Performance na Fonte: realizar a lógica no SQL economiza memória RAM no seu ambiente de análise e acelera o ciclo de insights.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

REFERÊNCIAS

BEAULIEU, Alan. *Learning SQL: Generate, Manipulate, and Retrieve Data*. 3rd Edition. O'Reilly Media, 2020. (O guia definitivo para funções analíticas).

CELKO, Joe. *Joe Celko's SQL for Smarties: Advanced SQL Programming*. Morgan Kaufmann, 2014. (Para quem deseja entender a lógica matemática profunda por trás das subqueries).

MOLINARO, Anthony. *SQL Cookbook: Query Solutions and Techniques for All SQL Users*. O'Reilly Media. (Excelente para padrões de subqueries correlacionadas).

ORACLE CORPORATION. *MySQL 8.4 Reference Manual - Aggregate Functions and Window Functions*. Disponível em: dev.mysql.com. Acesso em: 10 mar. 2026.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

PALAVRAS-CHAVE

CTEs, SQL, MySQL, Agregações, Performance, OLAP, ETL.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com



PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com